

RAG and FastAPI Overview

FastAPI is a modern Python web framework used for building APIs and backend services. It is designed to be fast, easy to use, and highly efficient. FastAPI is built on top of Starlette for web functionality and Pydantic for data validation. One of the main advantages of FastAPI is its high performance. It can handle a large number of requests efficiently and is often compared with Node.js and Go in terms of speed. Developers use FastAPI to create REST APIs, microservices, machine learning backends, and data processing applications. FastAPI automatically validates incoming request data using Python type hints. This reduces development time and helps prevent common programming errors. The framework also generates interactive API documentation automatically using OpenAPI and Swagger UI.

Retrieval-Augmented Generation, commonly known as RAG, is an AI architecture that combines information retrieval with large language models. Instead of relying only on the knowledge stored in the model, a RAG system retrieves relevant information from external documents before generating a response. A typical RAG pipeline consists of document loading, text chunking, embedding generation, vector storage, retrieval, and answer generation. Documents are first loaded into the system and divided into smaller chunks. Embeddings are then created for each chunk and stored in a vector database. When a user asks a question, the system retrieves the most relevant chunks and provides them to the language model as context. Embeddings are numerical representations of text that capture semantic meaning. Similar pieces of text have embeddings that are located close to each other in vector space. This enables efficient similarity search and retrieval. Vector databases are optimized for storing and searching embeddings. Popular vector databases include Chroma, Pinecone, Weaviate, and Milvus. These databases support nearest-neighbor search algorithms that quickly identify the most relevant documents for a given query. Tokenization is the process of converting text into smaller units called tokens. Language models process tokens rather than raw text. The quality of tokenization can influence model performance, context length, and computational efficiency. Large Language Models are trained on massive amounts of text data. During training, the model learns patterns, grammar, reasoning structures, and relationships between words. During inference, the trained model generates responses based on the input prompt and the knowledge learned during training. Modern AI applications often combine FastAPI, vector databases, embeddings, and large language models to create intelligent assistants capable of answering questions from custom document collections.